

closure_evidence_20260925

BOB-245 closure evidence — 2026-09-25

Revision: 1 Last modified: 2026-09-25T00:00:00Z

Item: BOB-245 — “git pre-commit mutation-marker guard false-positive-blocks legitimate docs/test prose describing the §1.1 methodology”.

1. Root-cause confirmation (independent re-check, never trusted on faith)

scripts/git_hooks/pre-commit (tracked source; installed verbatim into .git/hooks/pre-commit by scripts/install_git_hooks.sh) did a naive, completely UNSCOPED grep for the literal string MUTATED (assembled via string concatenation as M1="MUT" "ATED", plus two sibling markers) across **every single staged file**, with zero ability to distinguish:

- a genuine leftover mutation-test artifact accidentally left in shipped production code (a real defect this hook is supposed to catch), from
- legitimate PROSE in documentation/QA-evidence files, or legitimate COMMENTS in test source files, correctly and intentionally describing this project’s own MANDATORY §1.1 paired-mutation-testing methodology.

Confirmed real false-positive instances that blocked an otherwise-clean commit this session (each verified directly by reading the flagged line, not re-derived from a prior claim):

```
$ grep -n "UNMUTATED\|MUTATED" docs/qa/BOB-187/
closure_evidence_20260925.md
83:against the UNMUTATED sandbox copy, confirming it reproduces
6c's exit-2
```

```
$ grep -n "MUTATED" docs/qa/BOB-196/closure_evidence_20260925.md
232:MUTATED_RC=1
```

```
$ grep -n "UNMUTATED\|MUTATED" tests/unit/
test_ownership_precondition.sh
642:    # ---- (i) GREEN on the UNMUTATED sandbox copy: the
sandbox reproduces
677:    # ---- (iii) RED on the MUTATED sandbox copy, a
FRESH escape target
```

All five confirmed as real false positives — none is a genuine mutation residue leftover; every one is either documentation prose describing a reproduction, a documented shell variable name inside a code fence, or a comment in a test file describing the standard GREEN-on-unmutated / RED-on-mutated paired-test pattern this project's constitution mandates (§1.1). The .html twins of the two .md files carry the same text (confirmed at lines 271 and 461 respectively).

2. Sibling gate investigated (the correct pattern learned from)

scripts/pre_build/check_cm_no_production_mutation_residue.sh already solves this exact problem correctly, and its own header documents two PRIOR false-positive/false-negative iterations it learned from (BOB-070). Key techniques adopted for this fix:

1. **Scope restriction** to first-party PRODUCTION source roots (download-proxy, qBitTorrent-go, scripts, plugins, webui-bridge.py), explicitly excluding tests/, docs/, qa-results/, scratchpad/, constitution/, submodules/, challenges/, mutants/, .git/, vendored (node_modules/, .venv/, site-packages/), and build-output directories.
2. A fenced guard""rails:allow <reason> waiver mechanism (comment-only, reason mandatory, always printed/counted).
3. Structural parsing (masking string-literal interiors, tracking docstring/here doc regions) to distinguish a comment from a string literal from real code.

3. Fix applied

scripts/git_hooks/pre-commit was extended with three new scope-check helper functions — `_pc_is_excluded_path`, `_pc_is_scannable_type`, `_pc_is_production_path` — applied to each staged file BEFORE the existing marker grep runs:

- `_pc_is_production_path` requires the staged path to start with one of `download-proxy/`, `qBitTorrent-go/`, `scripts/`, `plugins/`, or be exactly `webui-bridge.py`.
- `_pc_is_excluded_path` (checked inside `_pc_is_production_path`) vetoes `tests/`, `docs/`, `qa-results/`, `scratchpad/`, `constitution/`, `submodules/`, `challenges/`, `mutants/`, `.git/`, `node_modules/`, `.env/`, `venv/site-packages/`, `__pycache__/`, and `out/build/dist/` — mirroring the sibling gate's exclusion set.
- `_pc_is_scannable_type` further restricts to `*.sh/*.bash/*.py/*.go` (matching the sibling gate's own file-type filter), so a documentation file that happens to sit inside a production root (e.g. a README under `scripts/`) is still out of scope.

A staged file is grepped for the mutation markers **only** if it passes `_pc_is_production_path`. The existing self-avoidance technique (marker tokens built via string concatenation) is preserved unchanged, and the new in-source comments explaining the fix were deliberately worded to never literally spell the marker string themselves (verified below).

No fenced waiver escape was added to this hook (unlike the sibling gate). Judgment call, documented in-source: the scope restriction alone eliminates every currently-known false-positive class for this commit-time hook, and production sources have no legitimate reason to inline-document the mutation-testing methodology (that documentation correctly lives in `tests/docs`, both already out of scope). Adding waiver-parsing logic to a hook that runs on every single commit would add complexity with no corresponding benefit today. The in-source comment points at the sibling gate's guard "`rails:allow <reason>`" mechanism as the extension point if a genuine future need arises.

4. Self-avoidance re-verification (the fix's own source)

```
$ grep -n "MUTATED" scripts/git_hooks/pre-commit
(no output – exit 1)
```

```
$ bash -n scripts/git_hooks/pre-commit
(clean – exit 0)
```

The fixed hook's own added prose describes the false-positive class without literally spelling the all-caps marker (using “pre-mutation” / “post-mutation” phrasing instead), so the fix's own source cannot become a sixth false positive against itself when this commit stages it.

5. New test: tests/hooks/ test_pre_commit_mutation_scope.sh

A new hermetic executing test (following the established tests/hooks/test_unattributed_commit_guard.sh pattern — a throwaway mktemp -d git repository, never the real project repo) drives the REAL scripts/git_hooks/pre-commit invocation path. Seven assertions:

1. bash -n parse sanity.
2. Fixture-genuinely-contains-the-marker sanity check (proves the test below is non-vacuous).
3. **RED-then-GREEN (the fix under test):** a file staged under docs/ containing the literal marker in legitimate prose — the hook now exits 0.
4. The same false-positive class under tests/ (comments describing the paired-mutation methodology) also exits 0.
5. **Negative control (proves the fix is a genuine narrowing, not a disable):** a file staged under scripts/ (a production root) containing a genuine residue-shaped marker still exits 1.
6. The refusal output names the offending production file.
7. A clean file with no markers anywhere still exits 0.

Real run (GREEN, current fixed source)

```
$ bash tests/hooks/test_pre_commit_mutation_scope.sh
PASS: hook source is syntactically valid bash
PASS: fixture doc genuinely contains the literal marker (test is
non-trivial)
PASS: staged docs/ file containing the literal marker in
legitimate prose: hook now exits 0 (RED-then-GREEN: this is the
fix)
PASS: staged tests/ file with legitimate mutation-methodology
comments: hook exits 0
PASS: staged scripts/ file with a genuine residue-shaped marker:
hook STILL exits 1 (negative control – real detection preserved)
PASS: refusal output names the offending production file
PASS: staged clean file with no markers: hook exits 0
```

=== Result: 7 passed, 0 failed ===

Paired §1.1 mutation (manually applied, confirmed RED, then reverted)

The scope-gate call site was neutered in a scratch copy of the fixed hook (`_pc_is_production_path "$file" || continue` → commented out, restoring the pre-fix unscoped behaviour):

```
$ sed -i 's/ _pc_is_production_path "\$file" || continue/ #
MUTATED-FOR-TEST: .../' scripts/git_hooks/pre-commit
$ bash -n scripts/git_hooks/pre-commit
mutant syntax OK
$ bash tests/hooks/test_pre_commit_mutation_scope.sh
  PASS: hook source is syntactically valid bash
  PASS: fixture doc genuinely contains the literal marker (test is
non-trivial)
  FAIL: staged docs/ file containing the literal marker in
legitimate prose: hook exited 1 (expected 0) - ...
  FAIL: staged tests/ file with legitimate mutation-methodology
comments: hook exited 1 (expected 0) - ...
  PASS: staged scripts/ file with a genuine residue-shaped marker:
hook STILL exits 1 (negative control – real detection preserved)
  PASS: refusal output names the offending production file
  PASS: staged clean file with no markers: hook exits 0
```

=== Result: 5 passed, 2 failed ===

RED confirmed: removing the scope-gate reproduces exactly the reported false-positive behaviour (Tests 3/4 flip to FAIL), proving the test genuinely catches a regression to the pre-fix behaviour and is not a bluff. The mutation was then reverted (restored from a pre-mutation backup) and diffed byte-identical to the pre-mutation source, and the 7/7 GREEN result was re-confirmed:

```
$ diff /tmp/pre-commit.orig.bak scripts/git_hooks/pre-commit
(empty – RESTORED_IDENTICAL)
$ bash -n scripts/git_hooks/pre-commit
syntax OK
$ bash tests/hooks/test_pre_commit_mutation_scope.sh
=== Result: 7 passed, 0 failed ===
```

6. Verification against the original 5 flagged files

The fixed hook was reinstalled from tracked source via the sanctioned install script, confirmed byte-identical to the installed copy, then the original 5 flagged files were staged and the INSTALLED hook was invoked directly:

```
$ bash scripts/install_git_hooks.sh
Installing git hooks from ../scripts/git_hooks to ../.git/
hooks ...
    Installed: pre-commit
    Installed: pre-push
    Installed: commit-msg
    Installed: post-commit
Done — 4 hook(s) installed. Constitution §11.4.75 enforcement
active.
$ diff scripts/git_hooks/pre-commit .git/hooks/pre-commit
(empty — INSTALLED_MATCHES_SOURCE)

$ git add docs/qa/B0B-187/closure_evidence_20260925.md docs/qa/
B0B-187/closure_evidence_20260925.html \
    docs/qa/B0B-196/closure_evidence_20260925.md docs/qa/
B0B-196/closure_evidence_20260925.html \
    tests/unit/test_ownership_precondition.sh
$ bash .git/hooks/pre-commit
$ echo "HOOK_EXIT=$?"
HOOK_EXIT=0
```

No false-positive output, exit 0 — the fix resolves all 5 originally reported false positives. The 5 files were then immediately unstaged (`git restore --staged ...`) — they belong to separate, already-in-flight work landing independently and were NOT committed as part of this fix:

```
$ git restore --staged docs/qa/B0B-187/
closure_evidence_20260925.md docs/qa/B0B-187/
closure_evidence_20260925.html \
    docs/qa/B0B-196/closure_evidence_20260925.md docs/qa/
B0B-196/closure_evidence_20260925.html \
    tests/unit/test_ownership_precondition.sh
(files return to their prior on-disk / untracked / unstaged-
modified state)
```

7. Toothiness preserved — the negative control, explained

The most important property of this fix is that it is a genuine **NARROWING** of scope, never a disable. Test 5 above proves this concretely: a file staged under `scripts/` (one of the five production roots) that contains an actual mutation-residue-shaped marker still trips the hook and blocks the commit with the offending file named in the output — the hook is exactly as strict as before for real production code, and only stopped grepping docs/tests/QA-evidence prose that was never the intended target in the first place.

8. Verdict

BOB-245 acceptance satisfied. Root cause confirmed independently, fix adopts the proven sibling-gate scope-restriction technique, self-avoidance preserved (including in the fix's own new prose), new hermetic test proves both the fix (RED→GREEN) and preserved detection (negative control) with a manually-applied and reverted paired §1.1 mutation, the fixed hook was reinstalled and directly verified against all 5 originally reported false positives with zero false-positive output, and the verification files were left untouched/unstaged afterward. Closing as Fixed.